

Task / Test	Status	Evidence	Notes
Session 1: Core three-model and observer comparison			
Capture true plaintext/no-TLS traffic on the wire	PASS	tcpdump capture on port 8000 showed the exact test token readable in plaintext	The token AURA-PLAINTEXT-NOTLS-2026-002 was visible in cleartext on the internal non-TLS hop between mitmproxy and the echo server.
Capture TLS-only traffic from a passive-observer perspective with Wireshark	PASS	Wireshark capture of localhost port 443 TLS traffic; packet-byte search showed the unique plaintext token was not present.	A passive observer could see TLS traffic and metadata, but not the plaintext token.
Confirm plaintext visibility at the TLS-terminating proxy with E2E off	PASS	mitmweb WebSocket capture showing the complete fake SSN/password prompt readable after TLS termination	Plaintext was visible. IMPLEMENTATION BUG: Server closed with code 4001 due to a malformed seq parsing error.
Capture a successful TLS-only plaintext echo round trip using the intended plaintext path	PASS	mitmweb shows the full plaintext request and echo response on /ws/plain; browser DevTools shows the readable echoed token returned to the client	Successful TLS-only round trip. The TLS-terminating proxy could read both directions of the application-layer plaintext.
Capture TLS + HPKE traffic from a passive-observer perspective with Wireshark	PASS	Wireshark capture of localhost port 443 traffic; packet-byte search returned "No packet contained" for AURA-WIRESHARK-E2E-2026-001	Passive observer could see TLS traffic and metadata, but not the unique plaintext token.
Confirm ciphertext visibility and prompt confidentiality at the TLS-terminating proxy with E2E on	PASS	mitmweb WebSocket capture showing encrypted ct content while the fake bank account and PRINCESS-DONUT-2026 recovery code are not readable	mitmproxy could observe the WebSocket traffic but could not read the prompt.
Capture the full encrypted round trip: ciphertext at mitmproxy and readable echo at the intended browser	PASS	Browser shows readable echoed token while mitmweb shows only encrypted ct content and does not reveal token	Intended browser received the readable echo successfully; the TLS-terminating proxy observed the traffic but could not read the token.
Create one observer-comparison summary covering plaintext, TLS-only, and TLS + HPKE	DONE	Markdown comparison summarizing what each observer could see across the three protection models	Plaintext was readable without TLS. TLS protected content from passive observation but exposed plaintext at TLS termination. TLS + HPKE kept prompt content hidden from the TLS-terminating proxy while the intended browser still received the readable echo.
Session 2: Identity, key-substitution, and handshake attacks			
Wrong server pin	PASS	Browser capture showing a non-canonical base64url variant of the pinned Ed25519 key was accepted as verified, followed by a second browser capture showing a genuine byte-level key mismatch was rejected and no E2E channel was established	Genuine key substitution was rejected as intended. SECURITY-RELEVANT VALIDATION WEAKNESS: Changing only the final base64url character from k to m produced a visibly different pin string that still verified because both strings decoded to the same 32-byte Ed25519 key. The browser therefore accepts visually different non-canonical encodings of the same cryptographic identity.
Modify the server Ed25519 identity key returned by /pubkey	PASS	Browser capture showing the substituted server identity key was rejected and no E2E channel was established	The proxy modified the server_ed25519 value returned by /pubkey. The browser rejected the substituted identity key and failed closed, preventing channel establishment.
Modify the server X25519 encryption key returned by /pubkey	PASS	Browser capture showing the substituted server encryption key was rejected and no E2E channel was established	The proxy modified the server_x25519 value returned by /pubkey. The browser rejected the tampered encryption key because the original signature no longer matched the modified key material and failed closed, preventing channel establishment.

Task / Test	Status	Evidence	Notes
Modify the /pubkey signature	PASS	Browser capture showing the tampered /pubkey signature was rejected and no E2E channel was established	The proxy modified the sig value returned by /pubkey while leaving the server keys unchanged. The browser rejected the invalid signature and failed closed, preventing channel establishment.
Tamper with server_hello handshake material	PASS	Browser capture showing the tampered server_hello handshake caused the encrypted channel to be torn down and prevented E2E establishment	The proxy modified the server_hello enc value in transit while leaving the original signature unchanged. The browser detected the invalid handshake material and failed closed by tearing down the encrypted channel.
Send msg before hello	PASS	Terminal capture showing a fresh WebSocket connection sent a msg frame before hello and the server closed the connection with code 4001	The server rejected the out-of-order msg frame while in AWAIT_HELLO state and closed the WebSocket with protocol error code 4001. No session was established and no plaintext fallback occurred.
Test malformed hello fields: missing/invalid enc, sig, or browser public keys	PASS	Terminal capture showing multiple malformed hello variants were rejected with WebSocket close code 4001	The server rejected hello frames with missing enc, missing sig, missing browser public keys, and invalid cryptographic values. Each malformed handshake failed closed with protocol error code 4001 and no session was established.
Send a second hello after the session is already established	PASS	Terminal capture showing a valid first hello established the session, then a second hello was rejected with WebSocket close code 4001	The server accepted the initial valid hello and returned server_hello, then rejected a second hello in the ESTABLISHED state and closed the connection with protocol error code 4001.
Send an unknown or incorrect frame type for the current protocol state	PASS	Terminal capture showing an unknown frame type was rejected with WebSocket close code 4001	The server rejected the bogus_frame type while in AWAIT_HELLO state and closed the connection with protocol error code 4001. No session was established.
Session 3: Ciphertext tampering and integrity attacks			
Change one character in a valid ciphertext	PASS	mitmproxy logs showing one-character ciphertext modification and connection termination	Changing one character in a valid ciphertext caused the message to fail without producing a valid plaintext echo.
Modify seq without modifying the corresponding ct	PASS	mitmproxy logs showing sequence number modification with ciphertext unchanged and connection termination	Changing the sequence number without modifying the corresponding ciphertext caused the message to fail without producing a valid plaintext echo.
Replace one message's ciphertext with another message's ciphertext	PASS	mitmproxy logs showing the first message ciphertext saved, substituted into the second message, and connection termination	Replacing one message's ciphertext with ciphertext from another message caused the message to fail without producing a valid plaintext echo.
Confirm tampered messages produce no valid plaintext or valid echo	PASS	Browser behavior and mitmproxy logs from ciphertext modification, sequence tampering, and ciphertext substitution tests	Across all ciphertext tampering tests, modified messages produced no valid plaintext and no valid echo.
Confirm tampering failures do not crash the server or expose sensitive internal information	PASS	Server logs showing generic tamper-failure messages and successful post-tamper encrypted echo	Tampering failures did not crash the server or expose plaintext, key material, stack traces, or other sensitive internal information; the server remained available and successfully echoed a subsequent valid encrypted message.
Session 4: Replay, ordering, and reflection attacks			
Replay an exact valid encrypted message	PASS	mitmproxy logs showing an exact valid encrypted message saved, replayed, and followed by connection termination without a duplicate echo	Replaying an exact valid encrypted message caused the connection to terminate without producing a duplicate plaintext echo.

Task / Test	Status	Evidence	Notes
Send the same sequence number twice	PASS	mitmproxy logs showing a second encrypted message changed to reuse the first sequence number and connection termination without a valid echo	Sending the same sequence number twice caused the second message to be rejected without producing a valid plaintext echo.
Send a sequence gap	PASS	mitmproxy logs showing a sequence number changed to create a gap and connection termination without a valid echo	Sending a sequence gap caused the out-of-sequence message to be rejected without producing a valid plaintext echo.
Send an old sequence number after a newer one	PASS	mitmproxy logs showing an older valid encrypted message injected after a newer valid message and connection termination without a duplicate echo	Sending an old valid encrypted message after a newer one caused the old message to be rejected without producing a duplicate plaintext echo.
Send encrypted messages out of order	PASS	mitmproxy logs showing the second encrypted message injected before the first and connection termination without a valid echo	Sending valid encrypted messages out of order caused the reordered traffic to be rejected without producing a valid plaintext echo.
Replay ciphertext from an earlier session into a new session	PASS	mitmproxy logs showing ciphertext saved from an earlier encrypted session, injected into a new session at the expected sequence number, and connection termination without a valid echo	Replaying ciphertext from an earlier encrypted session into a new session caused the message to be rejected without producing a valid plaintext echo.
Try using client-to-server ciphertext in the server-to-client direction	PASS	mitmproxy logs showing client-to-server ciphertext substituted into the server-to-client direction and connection termination without a valid echo	Using client-to-server ciphertext in the server-to-client direction caused the reflected ciphertext to be rejected without producing a valid plaintext echo.
Try using server-to-client ciphertext in the client-to-server direction	PASS	mitmproxy logs showing server-to-client ciphertext substituted into the client-to-server direction and connection termination without a valid echo	Using server-to-client ciphertext in the client-to-server direction caused the reflected ciphertext to be rejected without producing a valid plaintext echo.
Confirm rejected replay/ordering/reflection attacks do not produce duplicate echoes or plaintext	PASS	Browser behavior and mitmproxy logs from replay, ordering, cross-session replay, and reflection tests	Across all replay, ordering, cross-session replay, and reflection tests, rejected attacks produced no duplicate plaintext echoes and no valid plaintext output from the rejected traffic.
Session 5: Malformed-message and protocol robustness testing			
Send a msg missing ct	PASS	Standalone malformed-frame harness completed a valid handshake, sent a msg frame without the ct field, and captured the server rejection.	Server rejected the malformed frame with close code 4001 and reason malformed msg frame: 'ct'. No valid echo or plaintext was returned.
Send a msg missing seq	PASS	Standalone malformed-frame harness completed a valid handshake, sent a msg frame without the seq field, and captured the server rejection.	Server rejected the malformed frame with close code 4001 and reason malformed msg frame: 'seq'. No valid echo or plaintext was returned.
Test malformed seq values: numeric, empty, invalid hex, wrong length	PASS	Standalone malformed-frame harness completed a valid handshake for each case and tested numeric, empty, invalid-hex, and wrong-length seq values on fresh WebSocket connections.	Malformed seq values were rejected as intended. IMPLEMENTATION BUG: Numeric, empty, and invalid-hex sequence values returned explicit malformed-frame errors, while a wrong-length sequence value returned the generic close reason internal error, showing inconsistent error handling for malformed sequence length.
Send a frame missing type	PASS	Standalone malformed-frame harness completed a valid handshake, sent a frame without the type field, and captured the server rejection.	Server rejected the frame as intended with close code 4001 and reason expected msg in ESTABLISHED, got None. No valid echo or plaintext was returned.
Send an unknown type	PASS	Standalone malformed-frame harness completed a valid handshake, sent a frame with an unrecognized type value, and captured the server rejection.	Server rejected the unknown frame type as intended with close code 4001 and reason expected msg in ESTABLISHED, got 'not_a_real_type'. No valid echo or plaintext was returned.

Task / Test	Status	Evidence	Notes
Test malformed ciphertext encoding: empty, truncated, invalid Base64URL	PASS	Standalone malformed-frame harness completed a valid handshake for each case and tested empty, truncated, and invalid-Base64URL ciphertext values on fresh WebSocket connections.	Malformed ciphertext encodings were rejected as intended. IMPLEMENTATION BUG: The truncated ciphertext returned an explicit malformed-frame Base64 decoding error, while empty and invalid-Base64URL ciphertext values returned the generic close reason internal error, showing inconsistent error handling for malformed ciphertext input.
Send malformed/non-JSON input	PASS	Standalone malformed-frame harness completed a valid handshake for each case, sent both non-JSON text and malformed JSON on fresh WebSocket connections, and captured the server rejections.	Server rejected both malformed input cases as intended with close code 4001 and explicit malformed/non-JSON frame reasons. No valid echo or plaintext was returned.
Reproduce and classify the previously observed 4001 malformed seq issue	PASS	Standalone malformed-frame harness completed a valid handshake, resent a wrong-length seq value on a fresh WebSocket connection, and reproduced the same server response observed earlier.	The malformed sequence was rejected as intended. IMPLEMENTATION BUG: A wrong-length seq value consistently produced close code 4001 with the generic reason internal error rather than a specific malformed-frame validation message, reproducing the previously observed malformed-sequence issue.
Session 6: Plaintext leakage and logging checks			
Send a unique token with E2E off and search mitmproxy, server, client, and Docker logs	PASS	mitmproxy WebSocket capture showing the unique E2E-off token in plaintext and Docker log search showing no matching token in mitmproxy, client, or server container logs	With E2E disabled, the unique token was visible in plaintext to the intercepting proxy as expected, but was not found in mitmproxy, client, or server Docker logs. No unexpected plaintext logging was observed.
Send a different unique token with E2E on and search the same locations	PASS	mitmproxy WebSocket capture showing ciphertext-only encrypted message frames and Docker log search showing no matching plaintext token in mitmproxy, client, or server container logs	With E2E enabled, the unique token was absent from mitmproxy WebSocket traffic and from mitmproxy, client, and server Docker logs. Only ciphertext-bearing message frames were visible to the intercepting proxy, with no plaintext leakage observed.
Check browser console output for accidental plaintext leakage	PASS	browser developer console captured after sending a unique encrypted test token, showing no plaintext token or related sensitive message content	With E2E enabled, the unique test token did not appear in the browser console. No accidental plaintext leakage was observed in console output.
Check error paths for accidental plaintext or sensitive internal-detail leakage	PASS	Malformed sequence error path was reproduced with the standalone harness, and browser, server, and mitmproxy outputs were reviewed for plaintext and sensitive internal details.	Malformed input was rejected and no plaintext leakage was observed in the reviewed server or mitmproxy logs. SECURITY FINDING: One browser-visible 4001 error exposed Python-specific exception text (int() can't convert non-string with explicit base), revealing unnecessary internal implementation detail.
Summarize where plaintext is expected, observed, unexpectedly exposed, or absent	PASS	Review of the E2E-off and E2E-on mitmproxy captures, Docker log searches, browser console check, and malformed error-path reproduction.	Plaintext was expected and observed in mitmproxy with E2E disabled. With E2E enabled, the plaintext token was absent from mitmproxy traffic, container logs, and the browser console. SECURITY FINDING: One error path exposed Python-specific exception text, revealing unnecessary internal implementation detail despite no plaintext message leakage.
Session 7: Architecture, exposure, and residual visibility			

Task / Test	Status	Evidence	Notes
Attempt direct host access to localhost:3000	PASS	Direct curl attempt to http://localhost:3000 showing connection refused on both ::1 and 127.0.0.1.	Direct host access to the client service on localhost:3000 was blocked because the container port is not published to the host. The client remained reachable only through the intended proxied architecture.
Attempt direct host access to localhost:8000	PASS	Direct curl attempt to http://localhost:8000 showing connection refused on both IPv6 and IPv4.	Direct host access to the backend server on localhost:8000 was blocked because the container port is not published to the host. The backend remained inaccessible through direct host access.
Attempt direct access to the backend WebSocket	PASS	Direct WebSocket connection attempt to ws://localhost:8000/ws showing ConnectionRefusedError because the backend port was not published to the host.	Direct host access to the backend WebSocket was blocked. The connection attempt to ws://localhost:8000/ws was refused, confirming that the backend WebSocket was not directly reachable from the host.
Confirm mitmproxy is the only published ingress in the deployed Docker architecture	PASS	Docker container port mapping showing only mitmproxy with host-published ports, while the client and server expose internal container ports only.	Mitmproxy was confirmed as the only published ingress in the deployed Docker architecture. The client on port 3000 and backend server on port 8000 were not directly published to the host.
Record metadata still visible with HPKE on: timing, message count, ciphertext size, and connection information	PASS	Mitmproxy WebSocket and connection captures showing encrypted message frames while still exposing message count, exact timestamps, ciphertext and overall flow size information, and client/server connection details.	RESIDUAL RISK: With HPKE enabled, plaintext message content was not visible to mitmproxy. Observable traffic metadata remained, including four WebSocket messages, exact message timestamps, ciphertext and overall flow size information, SNI, TLS version and cipher, and client/server connection addresses. This is an expected limitation of payload encryption rather than an implementation failure.
Attempt to force an E2E downgrade	PASS	Browser capture showing the tampered server_hello caused the encrypted channel to be torn down, plus mitmproxy logs confirming the server_hello enc value was modified in transit.	An active intermediary modified the server_hello handshake material. The browser detected the fault, tore down the encrypted channel, and did not silently downgrade to plaintext.
Attempt to tamper with browser-side encryption code before the prompt is encrypted	PASS	Mitmproxy logs showing successful injection of client-side plaintext-capture code and recovery of the unique plaintext token before HPKE encryption.	SECURITY FINDING: The TLS-terminating intermediary successfully modified browser-delivered code and captured a unique plaintext token before HPKE encryption, while the browser still appeared healthy and the E2E channel remained established. HPKE itself was not broken; the result demonstrates that end-to-end confidentiality depends on the integrity of the browser-side encryption code and its delivery path.
Session 8: Final review and findings			
Review every completed test and confirm its final status	DONE	Completed review of all tests from Sessions 1–7, including final statuses, evidence references, and notes for expected behavior, security findings, implementation bugs, and residual risks.	All completed tests were reviewed and assigned a final status. No previously completed test remained unresolved. Findings and implementation issues were preserved in the notes for later classification.
Rerun only unexpected, failed, or unclear results	DONE	Reviewed all completed tests for unexpected, failed, or unclear outcomes and confirmed that previously questionable results had already been reproduced or clarified during Sessions 2, 5, and 6.	No additional reruns were necessary. Unexpected behaviors had already been independently reproduced or further tested, including the non-canonical server pin encoding, wrong-length sequence handling, malformed ciphertext errors, and browser-visible Python-specific exception text.

Task / Test	Status	Evidence	Notes
Separate implementation bugs from genuine security findings	DONE	Classification labels added to the relevant completed test notes.	Observed results were classified as genuine security findings, implementation bugs, residual risks, expected behavior, or successful security controls.
Select the strongest 3–5 findings and clearest exhibits for the paper and presentation	DONE	Reviewed all findings from Sessions 1, 2, 5, 6, and 7 and consolidated duplicate reproduction and summary rows into five unique issues with associated screenshots and logs.	Five unique red-team findings/issues were identified: acceptance of a non-canonical Base64URL encoding of the pinned Ed25519 identity key; inconsistent error handling for malformed sequence values, including a generic internal error for wrong-length input; inconsistent error handling for malformed ciphertext encodings; browser-visible exposure of Python-specific exception text; and successful client-code tampering by the TLS-terminating intermediary that captured plaintext before HPKE encryption while the browser still appeared healthy and the E2E channel remained established.